Note de séance SQL n°9

UPDATE, DELETE, ALTER TABLE, vues, index et EXPLAIN | 2026-10-27

1. Les notions

Notion	Règle à retenir
UPDATE / DELETE	Sans WHERE, toutes les lignes sont touchées. Prévisualiser avec un SELECT au même WHERE, travailler da
Transactions	START TRANSACTION puis COMMIT (valider) ou ROLLBACK (annuler). Les ALTER TABLE, TRUNCATE et
ALTER TABLE	ADD COLUMN, MODIFY (type), RENAME COLUMN, DROP COLUMN (définitif). Une colonne doit exister ava
Retirer des données	DELETE (lignes choisies, annulable en transaction) ; TRUNCATE (vide la table, irréversible, remet AUTO_INC
Mode sécurisé	SQL_SAFE_UPDATES = 1 refuse un UPDATE ou DELETE sans WHERE sur une clé : erreur 1175.
Vue	SELECT enregistré sous un nom : aucune donnée stockée, toujours à jour, définition unique. CREATE VIEW,
Index	Accélère la recherche, au prix de l'espace et de la lenteur des écritures. Créé automatiquement pour PRIMAR
EXPLAIN	type : ALL = table entière lue ; const, ref, range = index utilisé. key : index choisi. rows : lignes estimées.
Règles d'indexation	Indexer les colonnes des WHERE, JOIN, ORDER BY. Pas de fonction sur la colonne indexée (utiliser une pla

2. Préparation des copies de travail

```
USE formation;
SET SQL_SAFE_UPDATES = 0;
DROP TABLE IF EXISTS commandes_bac;
DROP TABLE IF EXISTS livraisons_bac;
CREATE TABLE commandes_bac LIKE commandes;
INSERT INTO commandes_bac SELECT * FROM commandes;
CREATE TABLE livraisons_bac LIKE livraisons;
INSERT INTO livraisons_bac SELECT * FROM livraisons;
```

3. Partie 1 : UPDATE, DELETE, ALTER TABLE

1 : Prévisualiser

```
SELECT id, produit, montant FROM commandes_bac WHERE montant < 50;
```

Casque 40, Souris 15, Clavier 30.

2 : UPDATE contrôlé par ROW_COUNT

```
UPDATE livraisons_bac SET statut = 'livrée', date_livraison = '2026-03-20'
WHERE commande_id = 6;
SELECT ROW_COUNT();                                    -- 1
SELECT * FROM livraisons_bac WHERE commande_id = 6;
```

3 : Transaction et arrondi

```
START TRANSACTION;
UPDATE commandes_bac SET montant = montant * 1.1 WHERE montant < 50;
SELECT * FROM commandes_bac WHERE montant < 50;      -- 44, 17, 33
ROLLBACK;
SELECT * FROM commandes_bac WHERE montant < 50;      -- 40, 15, 30
```

montant est un entier : 16,5 est arrondi à 17.

4 : ALTER TABLE ... MODIFY

```
ALTER TABLE commandes_bac MODIFY montant DECIMAL(10,2) NOT NULL;
DESCRIBE commandes_bac;
-- puis la même transaction : 44.00, 16.50, 33.00, puis 40.00, 15.00, 30.00
```

L'ALTER TABLE se lance avant la transaction : il n'est pas annulable.

5 : ADD COLUMN puis UPDATE avec CASE

```
ALTER TABLE commandes_bac ADD COLUMN categorie_montant VARCHAR(10);
UPDATE commandes_bac
SET categorie_montant = CASE WHEN montant < 100 THEN 'Petite'
                             WHEN montant < 300 THEN 'Moyenne'
                             ELSE 'Grande' END;
SELECT categorie_montant, COUNT(*) FROM commandes_bac GROUP BY categorie_montant;
```

Petite 3, Moyenne 2, Grande 2. « 300 et plus » est la condition, Grande est l'étiquette.

6 : UPDATE avec jointure

```
ALTER TABLE commandes_bac ADD COLUMN ville_client VARCHAR(50);
UPDATE commandes_bac o
INNER JOIN clients c ON o.client_id = c.id
SET o.ville_client = c.ville;
SELECT ROW_COUNT();                                  -- 6, juste après l'UPDATE
```

La commande 7 garde NULL (client inexistant).

7 : DELETE de la commande orpheline

```
SELECT o.* FROM commandes_bac o
LEFT JOIN clients c ON o.client_id = c.id WHERE c.id IS NULL;      -- prévisualisation
DELETE o FROM commandes_bac o
LEFT JOIN clients c ON o.client_id = c.id WHERE c.id IS NULL;
SELECT COUNT(*) FROM commandes_bac;                                -- 6
```

8 : Mode sécurisé et commandes irréversibles

```
SET SQL_SAFE_UPDATES = 1;
UPDATE commandes_bac SET montant = 0;                -- erreur 1175
SET SQL_SAFE_UPDATES = 0;
```

DELETE : retirer des lignes précises, annulable en transaction. TRUNCATE : vider une table rechargée régulièrement, sans retour possible. DROP TABLE : supprimer la table elle-même, structure comprise.

4. Partie 2 : les vues

1 et 6 : Vue de détail, puis ajout de la province

```
CREATE VIEW v_commandes_detail AS
SELECT o.id AS id_commande, o.date_commande, c.prenom AS client, c.ville,
       p.nom AS produit, p.categorie, o.montant
FROM commandes o
INNER JOIN clients c ON c.id = o.client_id
INNER JOIN produits p ON p.id = o.produit_id;

CREATE OR REPLACE VIEW v_commandes_detail AS
SELECT o.id AS id_commande, o.date_commande, c.prenom AS client, c.ville, v.province,
       p.nom AS produit, p.categorie, o.montant
FROM commandes o
INNER JOIN clients c ON c.id = o.client_id
INNER JOIN produits p ON p.id = o.produit_id
INNER JOIN villes v ON v.ville = c.ville;
```

6 lignes. Utiliser p.nom (et non la colonne texte o.produit). Vérifier le nom de la vue : une faute de frappe crée une seconde vue.

2 : Interroger la vue

```
SELECT ville, SUM(montant) AS total FROM v_commandes_detail GROUP BY ville;     -- Kinshasa 635, L
SELECT client, COUNT(id_commande) AS nb FROM v_commandes_detail GROUP BY client;
```

3 : Vue qui protège

```
CREATE VIEW v_clients_public AS
```

```
SELECT id, prenom, ville FROM clients;
```

8 lignes, 3 colonnes : ni l'âge ni le téléphone.

4 : Vue avec agrégation

```
CREATE VIEW v_depenses_client AS
SELECT c.id, c.prenom, COUNT(o.id) AS nb_commandes,
       COALESCE(SUM(o.montant), 0) AS depense_totale
FROM clients c
LEFT JOIN commandes o ON c.id = o.client_id
GROUP BY c.id, c.prenom;

SELECT * FROM v_depenses_client WHERE depense_totale > 250;
```

Awa, Jean, Grace.

5 : Une vue est toujours à jour

```
CREATE VIEW v_total_bac AS SELECT SUM(montant) AS total FROM commandes_bac;
START TRANSACTION;
UPDATE commandes_bac SET montant = montant + 100 WHERE id = 1;
SELECT * FROM v_total_bac;                          -- +100
ROLLBACK;
SELECT * FROM v_total_bac;                          -- valeur de départ
```

La vue exécute sa requête sur les données du moment.

7 : Gérer les vues

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
DROP VIEW v_total_bac;
```

Une vue ne stocke que sa définition (presque pas d'espace) et reste à jour ; une table copie duplique les données et se périme.

5. Partie 3 : index et EXPLAIN

Préparation : Table de test de 100 000 lignes

```
SET SESSION cte_max_recursion_depth = 100000;
DROP TABLE IF EXISTS commandes_test;
CREATE TABLE commandes_test (
    id INT PRIMARY KEY, client_id INT NOT NULL,
    montant DECIMAL(10,2) NOT NULL, date_commande DATE NOT NULL
);
INSERT INTO commandes_test
WITH RECURSIVE n AS (SELECT 1 AS i UNION ALL SELECT i + 1 FROM n WHERE i < 100000)
SELECT i, 1 + (i * 7) % 5000, 10 + (i * 13) % 990,
       DATE_ADD('2025-01-01', INTERVAL (i % 365) DAY)
FROM n;
```

1 : Index existants

```
SHOW INDEX FROM livraisons;
SHOW INDEX FROM commandes;
```

PRIMARY et l'index de la clé étrangère, créés automatiquement. commandes.client_id n'a pas d'index (sa clé étrangère n'a pas pu être créée).

2-4 : Avant et après l'index

```
SELECT COUNT(*) FROM commandes_test;                                  -- 100000
SELECT COUNT(*) FROM commandes_test WHERE client_id = 1234;          -- 20
EXPLAIN SELECT * FROM commandes_test WHERE client_id = 1234;         -- type ALL, key NULL, rows ~
CREATE INDEX idx_test_client ON commandes_test (client_id);
EXPLAIN SELECT * FROM commandes_test WHERE client_id = 1234;         -- type ref, rows ~20
```

Le résultat ne change pas, seule la façon de chercher change.

5 : Le piège de la fonction

```
CREATE INDEX idx_test_date ON commandes_test (date_commande);
EXPLAIN SELECT * FROM commandes_test WHERE MONTH(date_commande) = 3;     -- ALL, ~100000
EXPLAIN SELECT * FROM commandes_test
WHERE date_commande >= '2025-03-01' AND date_commande < '2025-04-01';    -- range, ~8500
```

Les deux donnent 8494 lignes. L'index est trié sur la date complète ; MONTH() oblige à calculer le mois de chaque ligne.

6 : Clé primaire et colonne sans index

```
EXPLAIN SELECT * FROM commandes_test WHERE id = 500;            -- const, 1 ligne
EXPLAIN SELECT * FROM commandes_test WHERE montant = 500;       -- ALL
```

7 : Réflexion

(a) Chaque index prend de l'espace et doit être mis à jour à chaque écriture. (b) Indexer selon les requêtes : client_id et date_commande de commandes ; id, produit_id et commande_id le sont déjà ; telephone et transporteur seulement si on cherche souvent dessus. (c) LIKE 'Ka%' utilise l'index (début connu, valeurs voisines dans l'ordre alphabétique) ; LIKE '%Ka' non.

8 : Nettoyage

```
DROP TABLE commandes_test;
```

6. Pièges rencontrés

• **ROW_COUNT()** doit être lancé juste après la commande ; COUNT(*) compte les lignes de la table, pas les lignes modifiées.

• **Condition et étiquette** : « 300 et plus Grande » = condition « 300 et plus », étiquette « Grande ». Si la lecture oblige à contourner une contrainte, relire l'énoncé.

• **CREATE OR REPLACE VIEW** avec une faute de frappe crée une nouvelle vue : vérifier avec SHOW FULL TABLES.

• **Colonne redondante dans une vue** : utiliser p.nom, pas la colonne texte o.produit, qui peut disparaître.

• **Index automatiques** : PRIMARY KEY, FOREIGN KEY et UNIQUE en créent ; inutile de les doubler.

• **Fonction sur une colonne indexée** : l'index est ignoré ; préférer une plage.

• **Coût des index** : espace et ralentissement des écritures ; on n'indexe pas tout.